

Project Design Document — Enterprise QA Automation

Generated: 26/7/2026, 6:57:59 pm

Executive Summary

This document describes the architecture, key components, and operational guidance for the Enterprise QA Automation framework used to validate banking workflows. It is intended for engineering interview discussions and operational handoffs.

Scope & Objectives

- Automate critical banking end-to-end workflows (login, account view, transfers, reconciliation).
- Support tag-based test selection (smoke, regression, api, ui).
- Enable multi-client matrix execution for cross-client compatibility.
- Produce reliable reports and CI integration.

Architecture Overview

The framework uses Playwright Test with TypeScript, organized around Page Objects, shared fixtures, and environment-driven configuration. Tests live under `src/tests` and are tagged to allow focused runs. Reporting uses Allure and a custom summary generator.

Key Components

- ``playwright.config.ts``: Central runner configuration and environment loader.
- ``src/pages``: Page Object Model implementations for UI stability.
- ``src/fixtures``: Shared Playwright fixtures (runtime config, clients, APIs).
- ``src/testdata``: Customer profiles and client matrix for multi-client runs.
- ``scripts/generate-report-summary.mjs``: Aggregates Allure results into human-readable summaries.

Tagging & Suites

Tests use tags such as `@smoke`, `@regression`, `@api`, and `@ui`. NPM scripts provide convenient entry points (e.g., ``npm run test:smoke``, ``npm run test:matrix``). CI triggers accept inputs for environment and suite selection.

Multi-Client Matrix

A client configuration file drives matrix runs. Regression suites iterate the configured clients to validate behavior across banks/environments. The fixtures expose resolved client config to test code for per-client parametrization.

Reporting & CI

Allure provides detailed test reports; a small summary script converts ``allure-results`` into Markdown and JSON for quick review and CI step summaries. CI workflow uploads artifacts and supports manual dispatch with environment and suite inputs.

How to Run Locally

- 1) Install dependencies: ``npm install``.
- 2) Run a suite: ``npm run test:smoke`` or ``npm run test:matrix``.
- 3) Generate report summary: ``npm run report:summary``.

Operational Notes

- Ensure ``env.{environment}`` files exist for the target environment.
- Keep POM locators and API client contracts stable; prefer selectors with `data-test` attributes.

- Use tag expressions for selective runs to shorten feedback loops.

Next Steps & Recommendations

- Produce a `docx` export if Word-native editing is required; this can be generated from the PDF or by a small script using `python-docx`.
- Add nightly matrix runs in CI to surface client-specific regressions early.
- Store generated reports in a centralized artifacts bucket for auditability.

Contact & Ownership

Owner: QA Automation Team. For questions or updates, open a PR or raise an issue in the repository.